Task Scheduler with a Max-Heap

CS314 Heap Sort - Mini Project Report

1. Problem

An operating system must always run the most important ready task next, while new tasks keep arriving. This needs a **priority queue** with two operations, INSERT and EXTRACT-MAX. This project builds a max-heap from scratch, uses it to simulate a priority scheduler, and measures how it compares with simpler ways to keep tasks ordered.

2. Design

The heap is a complete binary tree stored in a Python list (0-based): parent(i) = (i-1)//2, left(i) = 2i+1, right(i) = 2i+2. Every parent is >= its children, so the maximum is always at index 0. Implemented in `max_heap.py`: INSERT (append, then sift up), EXTRACT-MAX (move the last item to the root, then MAX-HEAPIFY down), BUILD-MAX-HEAP (heapify from the last parent upwards) and heap sort.

Operation	Time	Why
peek (maximum)	O(1)	the root
INSERT	O(log n)	sift up at most the tree height
EXTRACT-MAX	O(log n)	sift down at most the tree height
BUILD-MAX-HEAP	O(n)	most nodes are near the leaves and move very little
heap sort	O(n log n), O(1) extra space	n x EXTRACT-MAX in place

3. The scheduler

Each `Task` has a name, a priority (bigger = more important) and a duration. The scheduler stores `(priority, -arrival_number, task)` in the heap. The unique arrival number makes tasks with equal priority run first-come-first-served, and means Task objects are never compared. Running the 7 tasks of `python scheduler.py` (all arrive at time 0, non-preemptive) gives:

Order	Task	Priority	Duration	Start	Finish
1	kernel interrupt	10	1	0	1
2	video call	9	4	1	5
3	virus scan	4	6	5	11
4	email sync	3	2	11	13
5	print report	3	3	13	16
6	backup	2	8	16	24
7	system update	1	10	24	34

The two urgent tasks start at times 0 and 1, and the two equal-priority tasks (email sync, print report) keep their arrival order. In this example the average waiting time is 10.0 against 17.9 for plain first-come-first-served order; this depends on the example, because here the urgent tasks also happen to be short.

4. Testing

15 unit tests (`python -m unittest -v`) cover: empty heap errors, single items, the heap property after every INSERT and EXTRACT-MAX, the textbook BUILD-MAX-HEAP example, 2,000 random operations checked against a reference list, heap sort against `sorted()`, priority order, FIFO ties, and interleaved add/next calls. All pass.

5. Experiments

Experiment 1. n INSERTs followed by n EXTRACT-MAXs of random priorities with four strategies: our heap, an unsorted list (scan for the maximum), a sorted list (`bisect.insort`) and Python's `heapq` (written in C, as a reference). Time is the best of 3 runs (1 run above n = 10,000); every result is checked against a sorted copy. The unsorted list is skipped at n = 100,000 because it is quadratic.

n	our heap	unsorted list	sorted list	heapq (C)
10,000	21.6 ms	819.7 ms	6.9 ms	3.3 ms
30,000	82.1 ms	7.89 s	59.1 ms	15.0 ms
100,000	408.8 ms	skipped	556.8 ms	63.4 ms

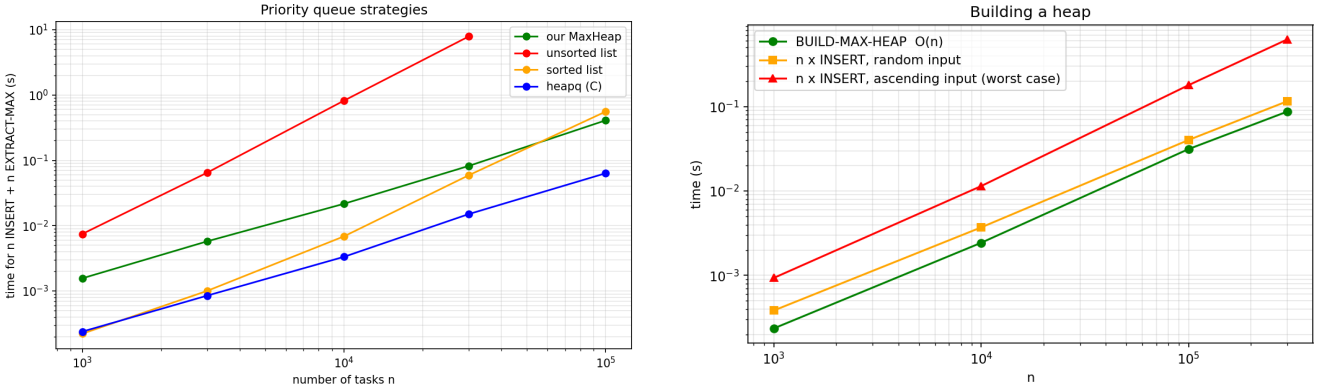


Figure 1 (left): Experiment 1. Figure 2 (right): Experiment 2. Both axes are logarithmic.

Experiment 2. BUILD-MAX-HEAP against n separate INSERTs. At n = 300,000: build 87.1 ms, INSERT of random data 115.5 ms, INSERT of ascending data (the worst case, every item sifts up to the root) 622.5 ms.

6. Analysis

The unsorted list is quadratic. From n = 10,000 to 30,000 its time grew 9.6x (3x more tasks, about 9x more time) and at n = 30,000 it took 7.89 s against 82.1 ms for the heap, about 96x slower. Each EXTRACT-MAX scans the whole list.

The heap scales like n log n. From n = 10,000 to 100,000 its time grew 19x, far below the 100x a quadratic method would need.

The sorted list is a surprise. It was faster than our heap up to n = 30,000 (59.1 ms vs 82.1 ms) but slower at n = 100,000 (556.8 ms vs 408.8 ms). Its INSERT is O(n), but the shifting is done by fast C code with a tiny constant, so the quadratic cost shows only for larger n. Big-O tells how time grows, not which method wins at a small size.

Our heap vs heapq. The C implementation was 6.5x faster at n = 100,000 with the same algorithm; the gap comes from Python's interpreter overhead, not from the idea.

BUILD-MAX-HEAP. On random data repeated INSERT was only about 1.3x slower, because a random item usually sifts up only a step or two (average O(1)). On ascending data it was 7.1x slower, which shows the O(n log n) worst case against the O(n) build.

7. Conclusion

A max-heap gives an O(log n) INSERT and EXTRACT-MAX, which is the right trade-off for a scheduler that keeps receiving tasks, and it avoids the quadratic cost of scanning a list. For small queues a sorted list can be just as fast in practice, and in real programs the library `heapq` is the practical choice. Possible extensions: preemption and task arrival times, priority aging so low-priority tasks do not starve, and updating a priority with a decrease/increase-key operation.